From PQC to HHE: Reusing a Co-Design Platform for Side-Channel-Protected PASTA

Ahmet Malal[1,2][0009−0009−4019−8617], Tolun Tosun[3,4][0000−0002−7665−5324], Oğuz Yayla[2][0000−0001−8945−2780], and Erkay Savaş[3][0000−0002−4869−5556]

[1] ASELSAN Inc., Ankara, Türkiye
[2] Middle East Technical University, Ankara, Türkiye
[3] Sabanci University, Istanbul, Türkiye
[4] enCRYPTON Labs, Istanbul, Türkiye

Abstract. Hybrid homomorphic encryption (HHE) lets a constrained client send compact symmetric ciphertexts while a server transciphers them into homomorphic ciphertexts, making HE-friendly ciphers such as PASTA a practical choice. Efficient and side-channel-secure execution of PASTA on embedded devices, however, remains challenging, since existing hardware relies on dedicated cipher cores and provides no side-channel protection. We present a hardware/software co-design of PASTA on RISQrypt, an existing post-quantum cryptography (PQC) platform, without adding any cipher-specific RTL. The affine layers, S-boxes, and SHAKE128 sampling are executed by the platform's arithmetic and Keccak accelerators under software control. We further integrate first-order arithmetic masking by reusing the masking accelerator for share refreshing and masked-multiplication randomness, requiring no additional hardware while keeping arithmetic on secret shares off the processor datapath. The masked multiplication shows no first-order leakage in a TVLA evaluation with 100 000 traces on an Artix-7 FPGA. Compared with a software-only baseline on the same core, the co-design achieves speedups of 9.68× and 9.85× in the unmasked and masked configurations, respectively. To the best of our knowledge, this is the first side-channel-protected, hardware-accelerated implementation of PASTA.

Keywords: Hybrid Homomorphic Encryption · Transciphering · PASTA · Hardware/Software Co-Design · Arithmetic Masking · Side-Channel Protection · RISC-V

1 Introduction

Fully homomorphic encryption (FHE) allows a server to compute on encrypted data without learning the plaintext, making it attractive for privacy-preserving outsourced computation. Its main drawback is the cost incurred by the client: direct homomorphic encryption is computationally expensive, and HE ciphertexts are much larger than the underlying plaintexts. For embedded devices, this creates both computation and communication bottlenecks. Hybrid homomorphic

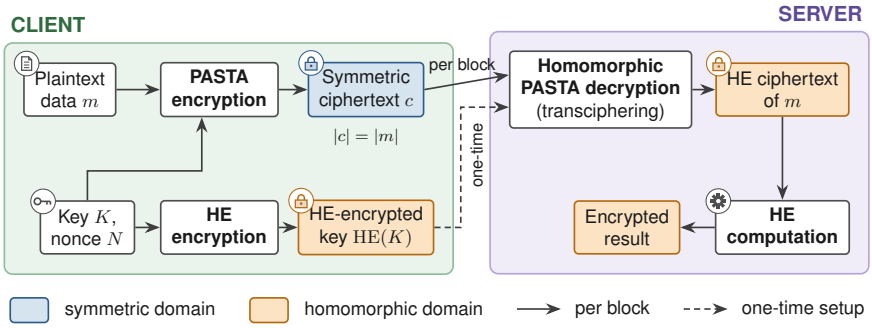


Fig. 1: HHE with PASTA-based transciphering. The client sends compact PASTA ciphertexts and an HE-encrypted key, while the server homomorphically evaluates the PASTA decryption circuit.

encryption (HHE), also known as transciphering, addresses this problem by having the client encrypt data with a symmetric cipher and transmit only compact symmetric ciphertexts together with a one-time HE encryption of the symmetric key. The server then homomorphically evaluates the symmetric decryption circuit to obtain an HE ciphertext of the data (Figure 1). This approach is practical only when the symmetric cipher is inexpensive to evaluate homomorphically, motivating the development of HE-friendly ciphers optimized for low multiplicative depth rather than conventional software performance [18,8,3,11,15,5]. Recent surveys provide a comprehensive overview of these designs and the remaining research challenges [21,20].

PASTA [12] is an HE-friendly stream cipher designed for the integer HE schemes BFV and BGV [13,4]. Its nonlinear layers have low multiplicative depth, while public affine layers provide diffusion, making it well suited for homomorphic evaluation. Nevertheless, efficient client-side execution remains challenging. Each encryption requires SHAKE128-based generation of public matrices and round constants, followed by large matrix-vector products over $\mathbb{F}_p$. Although these operations are conceptually simple, they dominate the execution time on embedded processors, making efficient and side-channel-secure implementations of PASTA an important practical problem.

Existing work addresses this challenge from complementary directions. The PASTA cryptoprocessor [2] implements a dedicated PASTA core with FPGA, ASIC, and RISC-V SoC realizations. It encrypts one PASTA-3 block in approximately 5,000 cycles, but occupies 65,468 LUTs, 36,275 FFs, and 256 DSPs on FPGA and provides no side-channel protection. PRESTO [16] similarly accelerates the CKKS-oriented ciphers HERA and Rubato using dedicated FPGA hardware without side-channel protection. On the security side, concurrent and independent work [1] studies power analysis of integer HHE ciphers and proposes a software masking framework evaluated on a Cortex-M4 microcontroller. However, prior work does not combine hardware acceleration and side-channel

protection on an embedded platform. This gap motivates our hardware/software co-design, which accelerates and protects PASTA by reusing existing platform accelerators without introducing a dedicated cipher core.

We address this gap with a hardware/software co-design of PASTA that reuses an existing post-quantum cryptography (PQC) platform. We implement PASTA on RISQrypt [22], a co-design platform originally developed for the lattice-based schemes Kyber and Dilithium, without adding any cipher-specific RTL.

Our contributions are as follows:

- **First hardware-accelerated masked PASTA.** To the best of our knowledge, this is the first side-channel protected PASTA implementation that runs on hardware accelerators. Prior masked PASTA work targets software on a general-purpose microcontroller [1], while ours is the first hardware/-software co-design of the cipher.
- **PQC platform reuse, zero new RTL.** We map PASTA onto RISQrypt [22], a co-design platform built for Kyber and Dilithium. The affine layers, S-boxes, and SHAKE128 sampling become sequences of existing accelerator commands, so a platform designed for lattice-based PQC also serves as an HHE client over $\mathbb{F}_p$ at no hardware cost.
- **Masking for free in area.** We reuse the platform's masking accelerator block for both mask refreshing and generating the randomness of the masked multiplication, so the protected design runs on the same blocks as the unmasked one. The masked arithmetic never reaches the processor datapath, which removes the software transition leakage that a masked microcontroller implementation must handle explicitly.
- **Validated and evaluated on FPGA.** We confirm the first-order security of the masked multiplication, the only nonlinear gadget in the design, with a TVLA evaluation on 100 K traces on an Artix-7 FPGA. Against a software-only baseline on the same core, the co-design is 9,68× (9,85×) faster in the unmasked (masked) case.

2 Background

2.1 Notation

All arithmetic is over the prime field $\mathbb{F}_p$, with $p = 65537$ in our instance. Field elements are lower case (a), vectors bold lower case ($\mathbf{x}$), and matrices bold upper case ($\mathbf{M}$), with $\mathbf{I}$ the identity. We write $\mathbf{a} \,\|\, \mathbf{b}$ for concatenation, $\mathbf{a} \odot \mathbf{b}$ for element-wise product, and $f \circ g$ for composition (g first). The state has $2t$ elements in two halves $\mathbf{x}_L, \mathbf{x}_R \in \mathbb{F}_p^t$, with r rounds indexed by j and branches by $k \in \{L, R\}$. A message $\mathbf{m} \in \mathbb{F}_p^l$ has l words in blocks of t. The nonce N and counter i select the keystream block, and the key is $\mathsf{sk} \in \mathbb{F}_p^{2t}$ with branch $\mathbf{k}$. A masked secret is split into two arithmetic shares, $\mathbf{a} = \mathbf{a}^{(0)} + \mathbf{a}^{(1)} \bmod p$, and fresh randomness is ρ (scalar) or $\boldsymbol{\rho}$ (vector).

2.2 Hybrid Homomorphic Encryption and Transciphering

In hybrid homomorphic encryption (HHE), first proposed in [19], the client encrypts its data with a symmetric cipher and sends the resulting compact ciphertext together with a one-time HE encryption of the symmetric key. The server then homomorphically evaluates the symmetric decryption circuit, converting the symmetric ciphertext into an HE ciphertext of the same data. This process, known as *transciphering*, allows the client to perform only symmetric encryption while the server continues computation on a standard HE ciphertext. Since the decryption circuit is evaluated homomorphically, the relevant metric is its homomorphic cost rather than software performance. Conventional ciphers such as AES [10] are therefore inefficient because their nonlinear layers incur high multiplicative depth. This has motivated the development of HE-friendly ciphers optimized for low multiplicative depth, with different designs targeting different HE schemes. Boolean schemes over $\mathbb{Z}_2$, such as TFHE, are paired with bit-oriented ciphers including FLIP [18], Trivium [5], and Elisabeth [9]. Approximate schemes over the reals, such as CKKS [7], use HERA [8] and Rubato [15], whereas integer schemes over $\mathbb{Z}_p$, such as BFV [13] and BGV [4], employ prime-field ciphers including LowMC [3], Rasta [11], and PASTA [12]. This work targets integer HE schemes.

2.3 PASTA Cipher

PASTA [12] is a family of stream ciphers over $\mathbb{F}_p$. It generates a keystream by applying the permutation PASTA-π to the secret key using a nonce N and block counter i, and truncating the result. The original proposal defines instances over 17-, 33-, and 60-bit prime fields. While these instances differ only in the underlying field, they also vary in the number of rounds r and block size t. We use PASTA-3 ($r = 3$, $t = 128$), which employs SHAKE128 and targets 128-bit security. Unless stated otherwise, PASTA refers to this instance throughout the paper.

Encryption and decryption. The secret key is sampled uniformly from $\mathbb{F}_p^{2t}$. Encryption splits the message into blocks of t elements and adds the keystream $\mathrm{left}_t(\text{PASTA-}\pi(\mathsf{sk}, N, i))$ to each block, where $\mathrm{left}_t(\cdot)$ denotes the first t elements. Decryption subtracts the same keystream. Since encryption and decryption execute the same permutation, they have identical computational cost; therefore, only encryption is described in the remainder of the paper.

The permutation. PASTA-π acts on $\mathbf{x} \in \mathbb{F}_p^{2t}$ and composes $r+1$ affine layers $A_{j,N,i}$ with r S-box layers, where the inner S-box layers are Feistel and the last one is cube,

$$\text{PASTA-}\pi(\mathbf{x}, N, i) = A_{r,N,i} \circ S_{\text{cube}} \circ A_{r-1,N,i} \circ S_{\text{feistel}} \circ \cdots \circ S_{\text{feistel}} \circ A_{0,N,i}(\mathbf{x}). \quad (1)$$

Figure 2 illustrates the permutation structure.

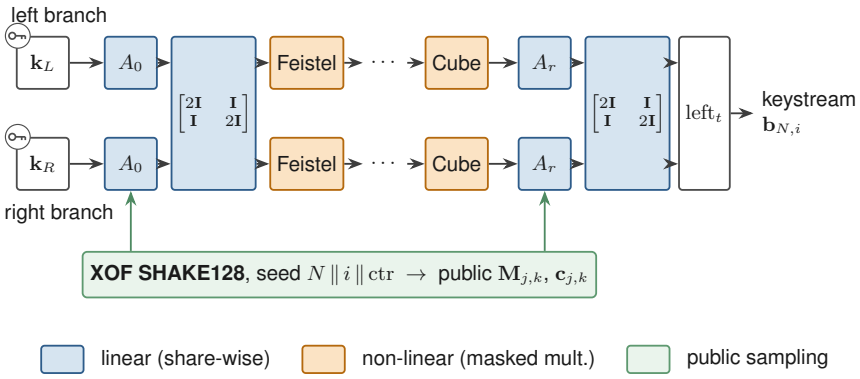


Fig. 2: The PASTA permutation. Public values from N and i build the affine layers, while the secret state passes through affine and nonlinear layers.

S-box layers. Both S-box layers act on each half of the state separately. The Feistel S-box adds the square of the previous element to each element, and the cube S-box raises every element to the third power. For a half $\mathbf{y} \in \mathbb{F}_p^t$ and index $l \in \{0, \ldots, t-1\}$,

$$[S_{\text{feistel}}(\mathbf{y})]_l = \begin{cases} y_0, & l = 0, \\ y_l + y_{l-1}^2, & \text{otherwise}, \end{cases} \qquad [S_{\text{cube}}(\mathbf{y})]_l = y_l^3. \tag{2}$$

These are the only nonlinear operations in the cipher, and they are the parts that need masked multiplication gadgets in Section 4.

Affine layer. Each affine layer applies a matrix product and a round-constant addition to each half, then mixes the two halves with a fixed block matrix,

$$A_{j,N,i}(\mathbf{x}) = \begin{bmatrix} 2\mathbf{I} & \mathbf{I} \\ \mathbf{I} & 2\mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{M}_{j,L,N,i}\,\mathbf{x}_L + \mathbf{c}_{j,L,N,i} \\ \mathbf{M}_{j,R,N,i}\,\mathbf{x}_R + \mathbf{c}_{j,R,N,i} \end{bmatrix}. \tag{3}$$

The matrices $\mathbf{M}_{j,k,N,i} \in \mathbb{F}_p^{t \times t}$ and round constants $\mathbf{c}_{j,k,N,i} \in \mathbb{F}_p^t$ are public. They are produced for each round by SHAKE128 seeded with N and i, so the server can regenerate them without any shared secret.

Sampling the matrices. Each $\mathbf{M}_{j,k,N,i}$ must be invertible. PASTA obtains this cheaply with a *sequential* matrix, whose top $t-1$ rows form a shifted identity and whose last row holds t random nonzero elements $\alpha_1, \ldots, \alpha_t \in \mathbb{F}_p \setminus \{0\}$. The matrix used in the affine layer is the transpose of this generator. Building it needs only t random elements, $t(t-1)$ multiplications, and $(t-1)^2$ additions, and the row-by-row structure is what our co-design exploits in Section 3.

2.4 The RISQrypt Platform

RISQrypt [22] is a hardware/software co-design platform built for the lattice-based schemes Kyber and Dilithium. A small RISC-V core runs the software that issues commands to three hardware blocks. The first is a programmable arithmetic accelerator called NTT-Lite. NTT-Lite is named after the number-theoretic transform, which lattice schemes use to multiply polynomials in $O(N \log N)$ instead of $O(N^2)$. PASTA does not use the NTT at all. What we use instead are the general coefficient-wise operations the accelerator also supports, listed below, so from here on we refer to NTT-Lite simply as the arithmetic accelerator to avoid confusion.

The second is a Keccak accelerator that provides SHAKE128. The third is the X2X accelerator, a mask-conversion and masking utilities block with its own pseudo-random number generator, which the platform uses for the masked variants of its PQC workloads. The name X2X originates from the so-called A2B and B2A conversion operations. However, in our work, the block is used exclusively for mask refreshing and for its pseudo-random number generator. Therefore, we refer to it as the masking accelerator throughout the remainder of the paper.

- `pwm`: element-wise product of two arrays, $c_k = a_k \cdot b_k$.
- `mac`: element-wise product accumulated onto the current result.
- `sum`: reduction of one array to a single field element, $s = \sum_k a_k$.
- `add`, `sub`: element-wise sum and difference of two arrays.
- `mul_const`: multiplication of every element by a constant fixed earlier by `set_bound`.
- `add_const`: loading of a constant array into the accelerator.

The accelerator can also forward the output of one command directly into the next without writing it back to memory, which we denote with the flags `OUTPUT_DIS` and `INPUT_DIS`. This chaining is what the memory optimization of Section 3.3 builds on.

2.5 Masking and Leakage Model

Masking is a common countermeasure against side-channel analysis. It splits each secret variable into $d + 1$ random shares. The security is analyzed in the *probing model*. Here an attacker who observes up to d intermediate values learns nothing about the secret. In *arithmetic masking* over $\mathbb{F}_p$, a secret is written as the sum of its shares. For first order, this is $a = a^{(0)} + a^{(1)} \bmod p$. Linear maps are cheap, since $L(a) = L(a^{(0)}) + L(a^{(1)})$. Each share is then processed on its own, with no fresh randomness. Multiplication is the hard case. The product mixes shares of both operands into cross-terms.

A common way to handle this is the two-share prime-field multiplication gadget of [6]. It computes the product $c = a \cdot b$ from the shares of a and b using fresh randomness ρ. The two output shares are formed as

$$c^{(0)} = a^{(0)}b^{(0)} - \rho, \qquad c^{(1)} = a^{(0)}b^{(1)} + \rho + a^{(1)}b^{(0)} + a^{(1)}b^{(1)}. \tag{4}$$

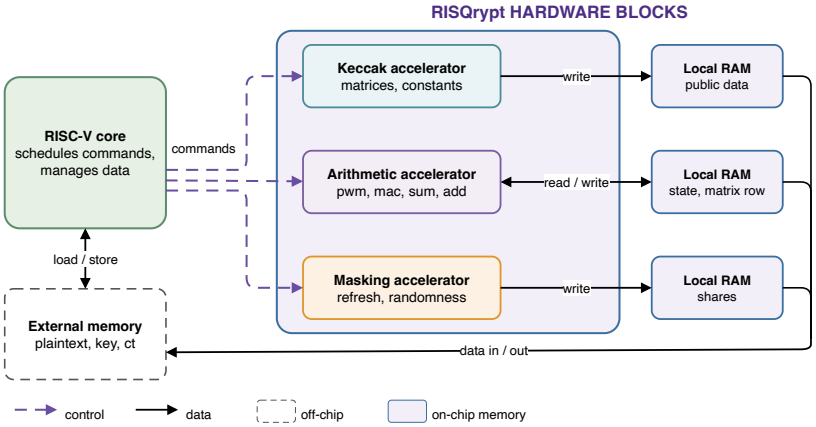


Fig. 3: Proposed PASTA co-design. The RISC-V core schedules commands, the Keccak accelerator generates public matrices and constants, and the arithmetic accelerator runs the field operations. The masked version also reuses the masking accelerator for refreshing and randomness.

The mask ρ is added before the cross terms $a^{(0)}b^{(1)}$ and $a^{(1)}b^{(0)}$ are accumulated. Thus no intermediate value ever forms the unmasked sum $a^{(0)}b^{(1)}+a^{(1)}b^{(0)}$, while the shares still reconstruct the correct product $c = c^{(0)} + c^{(1)}$.

3 Co-Design for PASTA

The proposed co-design implements PASTA by reusing the hardware blocks already available in RISQrypt, without introducing any cipher-specific RTL. As illustrated in Figure 3, the RISC-V core controls the execution, while the arithmetic and Keccak accelerators perform the computationally intensive operations. This mapping extends a platform originally designed for lattice-based PQC to an HHE cipher over $\mathbb{F}_p$. The processor supplies only commands, addresses, and public parameters; arithmetic on the cipher state remains within the hardware accelerators. Consequently, secret values do not enter the processor datapath, which also facilitates the masked implementation described in Section 4.

3.1 Mapping PASTA to RISQrypt

The arithmetic accelerator processes one branch of t field elements at a time; therefore, each operation is applied separately to the left and right branches. Table 1 summarizes the command sequence for each PASTA operation. Round-constant addition maps directly to `add`. The mixing layer configures the constant 2 through `set_bound` and applies one `mul_const` and one `add` per branch. The cube S-box requires two `pwm` commands, whereas the Feistel S-box requires a

shift followed by `pwm` and `add`. Key loading and matrix multiplication require additional consideration.

Table 1: Mapping of each PASTA operation to a sequence of arithmetic accelerator commands.

PASTA operation	Arithmetic accelerator command sequence
Load key ($\mathbf{y} \leftarrow \mathbf{k}$)	`set_bound(0)`, `add_const`
Matrix row $\times$ **branch**	`pwm`, `sum`
Next matrix row	`mul_const`, `add`
Add round constant	`add`
Mixing layer	`set_bound(2)`, `mul_const`, `add`
Cube S-box	`pwm`, `pwm`
Feistel S-box	shift, `pwm`, `add`

The key is transferred to the accelerator working memory without passing through a processor register. After configuring the constant as zero with `set_bound(0)`, `add_const` evaluates $\mathbf{y} = 0 \cdot \mathbf{y} + \mathbf{k} = \mathbf{k}$, thereby loading the key branch without a software copy.

Matrix-vector multiplication is the most expensive cipher operation. The sequential structure of the PASTA matrix allows it to be evaluated one output coordinate at a time. For coordinate i, `pwm` multiplies the current matrix row element-wise with the state branch, and `sum` computes

$$(\mathbf{My})_i = \sum_k \mathbf{M}[i,k] y_k.$$

Only the current row is stored; the next row is generated using one `mul_const` and one `add`. The matrix product therefore requires one `pwm` and one `sum` for each of the t output coordinates, whereas an S-box processes the entire branch with only one or two `pwm` commands.

3.2 Sampling and Round Constants

Every affine layer requires two public matrices and two public round constants, one pair per branch. Since these values depend only on the nonce N and block counter i, they are generated by the Keccak accelerator using SHAKE128.

Each object is derived from the 17-byte seed $N \,\|\, i \,\|\, \mathrm{ctr}$, where the one-byte counter ctr assigns an independent SHAKE128 stream to each matrix generator and round constant. The output is parsed as 64-bit words, and the low 17 bits form candidate elements of $\mathbb{F}_p$ for $p = 65537$. Rejection sampling retains candidates smaller than p; matrix generation additionally rejects zero to preserve invertibility. Since all seeds and sampled values are public, the variable number of rejections causes no secret-dependent timing.

Optimized sampling of matrix elements: For the matrix elements we offer a second option, controlled by the flag REJ_SAMP. With REJ_SAMP disabled, we take the low 16 bits of each word and add one,

$$a = \big(\text{low 16 bits}\big) + 1 \in [1,\, 2^{16}] = [1,\, p-1], \tag{5}$$

which needs no rejection at all and is constant-time by construction. This is exact for PASTA-3, because $p = 65537 = 2^{16} + 1$ maps the 2^{16} possible words one to one onto the nonzero field elements, so the distribution stays uniform. The round constants always follow the PASTA specification with 17-bit rejection, since they accept 0. We use this option to measure the cost of rejection sampling in Section 6.

3.3 Memory Optimization

The command sequence in Section 3.1 creates several intermediate vectors inside one PASTA round. Without chaining, each intermediate result would be written back through the software interface and then loaded again for the next accelerator command. This is costly for PASTA, because the state contains full branches of t field elements and the matrix product already invokes many short accelerator commands.

We therefore keep intermediate results in the accelerator-local memory whenever the next operation can consume them directly. In one round, this lets us chain the matrix product, round-constant addition, mixing, and S-box computation without unnecessary transfers through the processor. The state is moved only when the round schedule requires a new source or destination buffer. We denote this configuration by MEM_OPT in the performance results. This optimization preserves the PASTA computation while reducing data movement between accelerator commands. Its effect is measured in Section 6.

3.4 Software Layer

In the co-design, the RISC-V core coordinates the execution of the accelerator commands. It follows the PASTA round schedule, issues the required accelerator commands, and advances the counter used for SHAKE128-based sampling. The cipher state is processed by the hardware blocks, while the software provides only commands, addresses, and public parameters.

We also implement a software-only baseline that runs the same PASTA algorithm entirely on the RISC-V core, without using the arithmetic accelerator. To keep the comparison fair, SHAKE128 uses an assembly-optimized Keccak implementation adapted from [17], rather than a plain C reference. This baseline is used for the speed-up and memory-footprint comparisons in Section 6.

4 Masked PASTA

We now describe how the PASTA co-design is extended with first-order arithmetic masking. The masking model and the masked multiplication gadget were

```
void masked_poly_mult_mm(masked_poly *C,
                         const masked_poly *A,
                         const masked_poly *B) {
    poly r;
    x2x_prng_read(&r, N);    /* fresh mask rho */
    ntt_lite_set_clr_with_twiddle();
    /* c0 = a0*b0 - rho */
    ntt_lite_pwm(NTT_LITE_OUTPUT_DIS, A->share[0].coeffs, B->share[0].coeffs);
    ntt_lite_sub(C->share[0].coeffs, NTT_LITE_INPUT_DIS, r.coeffs);
    /* u = a0*b1 + rho   (mask added before the cross terms) */
    ntt_lite_pwm(NTT_LITE_OUTPUT_DIS, A->share[0].coeffs, B->share[1].coeffs);
    ntt_lite_add(NTT_LITE_OUTPUT_DIS, NTT_LITE_INPUT_DIS, r.coeffs);
    /* u += a1*b0 */
    ntt_lite_mac(NTT_LITE_OUTPUT_DIS, B->share[0].coeffs, A->share[1].coeffs);
    ntt_lite_set_clr_with_twiddle();
    /* c1 = u + a1*b1 */
    ntt_lite_mac(C->share[1].coeffs, A->share[1].coeffs, B->share[1].coeffs);
}
```

Listing 1.1: The masked multiplication of [6] expressed with the RISQrypt accelerator commands.

introduced in Section 2.5. This section describes their integration into the PASTA round function. The affine layers are processed share-wise, while the Feistel and cube S-boxes are implemented with the masked multiplication gadget and fresh randomness from the masking accelerator.

4.1 Masked Linear Layers

The affine layer of PASTA contains only linear operations over $\mathbb{F}_p$, namely matrix multiplication, round-constant addition, and branch mixing. Therefore, it can be evaluated share-wise. For a masked branch $\mathbf{x} = \mathbf{x}^{(0)} + \mathbf{x}^{(1)}$, each share is processed with the same public matrix $\mathbf{M} \in \mathbb{F}_p^{t \times t}$ of the affine layer,

$$\mathbf{y}^{(s)} = \mathbf{M}\mathbf{x}^{(s)}, \qquad s \in \{0, 1\}. \tag{6}$$

The same rule applies to the fixed mixing layer. Round constants $\mathbf{c} \in \mathbb{F}_p^t$ are public, so it suffices to add them to a single share,

$$(\mathbf{x}^{(0)} + \mathbf{c}) + \mathbf{x}^{(1)} = \mathbf{x} + \mathbf{c}. \tag{7}$$

Thus, the affine layer is masked by running the unmasked command sequence once per share, with no fresh randomness.

4.2 Masked Nonlinear Layers

The nonlinear layers of PASTA are the only steps that require masked multiplication. We use the gadget described in Section 2.5 coefficient-wise on vectors of t field elements. Listing 1.1 shows how it is expressed with the accelerator commands of the RISQrypt platform. The fresh mask ρ is drawn from the masking

accelerator's randomness source (line 5), all partial products run on the arithmetic accelerator through `pwm` and `mac`, and the mask is added before the cross terms are accumulated (lines 11–14), so no intermediate value ever forms the unmasked sum $a^{(0)}b^{(1)} + a^{(1)}b^{(0)}$. All operands remain within the accelerator, preventing the processor from combining the two shares.

Masked Feistel S-box. For a branch $\mathbf{y}$, the Feistel S-box computes

$$\mathbf{z} = \mathbf{y} + \mathbf{y}' \odot \mathbf{y}', \qquad \mathbf{y}' = (0, y_0, y_1, \ldots, y_{t-2}). \tag{8}$$

In the masked implementation, the shift is applied independently to both shares. The shifted value is refreshed using the masking accelerator and then squared with one masked multiplication (Listing 1.1). The final addition with $\mathbf{y}$ is linear and is done share-wise. The masked Feistel S-box therefore requires one masked multiplication.

Masked Cube S-box. For a branch $\mathbf{y}$, the cube S-box computes

$$\mathbf{z} = \mathbf{y} \odot \mathbf{y} \odot \mathbf{y}. \tag{9}$$

The masked cube S-box is evaluated using two masked multiplications. First, we compute the masked square

$$\mathbf{s} = \mathbf{y} \odot \mathbf{y}, \tag{10}$$

and then multiply it by the original masked value,

$$\mathbf{z} = \mathbf{s} \odot \mathbf{y}. \tag{11}$$

The operands are refreshed before each masked multiplication using the masking accelerator.

The masked permutation keeps the same round structure as the unmasked one. The affine layers run once per share with the unmodified command sequences, while the S-box layers are evaluated through the masked multiplications described above.

5 Side-Channel Leakage Evaluation

To evaluate the effectiveness of the proposed masking scheme, we conducted practical side-channel analysis (SCA) experiments on the masked PASTA implementation. Specifically, we employed the Test Vector Leakage Assessment (TVLA) methodology [14], which is widely used for leakage detection in the SCA literature. In the TVLA procedure, two sets of side-channel traces are collected from the target device. A trace corresponds to the side-channel measurements, such as power consumption, acquired while the device executes the cryptographic operation. In our experiments, the target device executes the masked PASTA implementation.

The first trace set, referred to as the *fixed-input* set, is collected using a constant secret input for all executions. The second set, referred to as the *random-input* set, is obtained by selecting the secret input independently and uniformly at random for each execution. The two trace sets are then compared using Welch's t-test to determine whether they exhibit statistically distinguishable leakage. If the masking scheme is implemented correctly, no statistically significant difference is expected between the *fixed-input* and *random-input* trace sets. For each sampling point, the TVLA computes a test statistic, referred to as the t-value. A commonly adopted decision threshold is an absolute t-value of 4.5. When the computed $|t|$ exceeds this threshold, the null hypothesis is rejected with a confidence level of approximately 99.999%. In this context, the null hypothesis states that the leakage distributions of the *fixed-input* and *random-input* trace sets are statistically indistinguishable.

We adopted the same SCA setup from [22]. The experiments were conducted using the ChipWhisperer framework. Trace acquisition was performed with the ChipWhisperer Husky[5], while the target device was a ChipWhisperer CW305[6] evaluation board equipped with a Xilinx Artix-7 FPGA (XC7A100T), which executed the proposed implementation. The target FPGA was operated at a core clock frequency of 40 MHz. The acquisition hardware was configured to record four samples per clock cycle. To validate the experimental setup, we additionally report the t-test results obtained with the pseudo-random number generator (PRNG) disabled. In this configuration, 1 K traces were collected, whereas 100 K traces were acquired for the experiments with the PRNG enabled.

We evaluated the masked multiplication described in Section 2.5 using the TVLA methodology. As discussed previously, masked multiplication constitutes the only nonlinear operation in the masked PASTA implementation, making its protection considerably more challenging than the remaining components, which are implemented using straightforward share-wise processing. The corresponding t-test results are shown in Figure 4. When the `PRNG_OFF` configuration is used, the observed t-values exceed the threshold with only 1 K traces. In contrast, under the `PRNG_ON` configuration, all t-values remain below the threshold of 4.5 across all sample indices. These results indicate that the proposed masked multiplication implementation successfully passes the TVLA evaluation.

6 Performance and Implementation Results

This section reports the cost of our implementations and compares the hardware/software co-design against the software-only baseline.All source code for our implementation is publicly available at `https://github.com/cisec-su/RISQrypt`[7]. The repository also contains the side-channel analysis (SCA) scripts

[5] `https://rtfm.newae.com/Capture/ChipWhisperer-Husky/`

[6] `https://rtfm.newae.com/Targets/CW305%20Artix%20FPGA/`

[7] The developed PASTA software layer has been merged into the original RISQrypt repository.

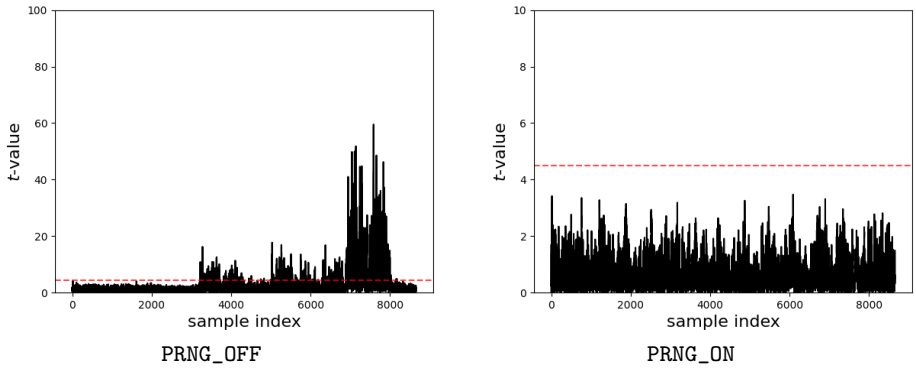


Fig. 4: TVLA results under two configurations: `PRNG_OFF` and `PRNG_ON`.

used for trace acquisition and testing in the experiments presented in the previous section.

We evaluate the proposed PASTA co-design against a software baseline for both masked and unmasked scenarios. We also separately evaluate two implementation options that affect the execution cost. The first is the memory optimization of Section 3.3, denoted MEM_OPT, which keeps intermediate results in accelerator-local memory. The second is rejection sampling, which is part of the public matrix and round-constant generation. The following subsections present each measurement in turn.

6.1 Experimental Setup

We run all implementations on the RISQrypt platform [22] by reusing its existing FPGA bitstream available in its open-source repository, which targets a Xilinx Artix-7 XC7A100T FPGA. We measure performance in clock cycles using the platform cycle counters, so the reported cycle counts are independent of the operating frequency. Each measurement corresponds to one PASTA-3 encryption block.

All software is compiled with the RISC-V toolchain using the `-O3` option. The software-only baseline and the co-design use the same PASTA parameters and the same SHAKE128 generation procedure. The masked implementations add the share-wise processing, masked multiplication gadgets, and fresh randomness described in Section 4.

6.2 Cycle Counts: Software vs. HW/SW Co-design

Table 2 reports the encryption cost of the two designs for a single block of PASTA, in an unmasked and a masked version. We give the cycle count of each design, the ratio of the software cost to the co-design cost in the column Speed-up, and the per-element and per-bit cost of each design.

Table 2: Encryption cost of PASTA-3 for one block, with the speed-up of the co-design over the software baseline and the per-element and per-bit cost. Per-bit figures use the 17-bit storage width of a field element.

		SW	**Co-design**	**cycles/element**		**cycles/bit**	
Masking	**Speed-up**	**(cycles)**	**(cycles)**	**SW**	**Co-design**	**SW**	**Co-design**
✗	9,68×	9.267.385	957.164	72.401	7.478	4.259	440
✓	9,85×	14.485.786	1.470.299	113.170	11.487	6.657	676

The acceleration mainly affects the PASTA permutation, which dominates the encryption cost. In the unmasked case, the software baseline needs 9.267.385 cycles, while the co-design needs 957.164 cycles, a speed-up of 9,68×. In the masked case, the software baseline needs 14.485.786 cycles and the co-design needs 1.470.299 cycles, a speed-up of 9,85×.

One block of PASTA-3 produces $t = 128$ field elements. We report costs as cycles per field element, which is the natural unit for a prime-field cipher and matches the metric used by the dedicated PASTA hardware [2]. In the unmasked case, the software baseline costs $9.267.385/128 \approx 72.401$ cycles/element, while the co-design costs 7.478. In the masked case, the software baseline costs 113.170 cycles/element, while the co-design costs 11.487.

For comparison with bit-oriented designs, we also provide a per-bit figure. Each element of $\mathbb{F}_p$ with $p = 65537$ is represented in 17 bits, so a block occupies $128 \times 17 = 2.176$ bits. Dividing the cycle count by this number gives the per-bit cost in the last two columns of Table 2. The unmasked co-design runs at about 440 cycles per bit, compared to 4.259 for the software baseline, and the masked co-design runs at about 676 cycles per bit, compared to 6.657 for the software.

Because PASTA works as a stream cipher, decryption runs the same permutation as encryption and differs only in the final addition or subtraction of the keystream, as described in Section 2.3. The numbers reported here for encryption therefore apply to decryption as well.

Key generation is not shown in the table. It is the same in software and in the co-design, since it only draws a random key and does no arithmetic on the accelerator, costing 1.281 cycles in every design.

6.3 Effect of Memory Optimization

Section 3.3 describes the memory optimization (MEM_OPT), which keeps intermediate results in the local RAM and removes most load and store calls between operations. We quantify its effect on the encryption cycle count. Table 3 compares the co-design with MEM_OPT enabled and disabled, for both the unmasked and the masked versions.

Turning the optimization off raises the unmasked encryption cost from 957.164 to 1.508.605 cycles, a slowdown of about 1,58×. For the masked version the cost rises from 1.470.299 to 2.295.773 cycles, a slowdown of about 1,56×. Since both

Table 3: Effect of the memory optimization on the encryption cost of the co-design (cycles for one block of PASTA-3).

Masking	MEM_OPT	Cycles	Slowdown
✗	✓	957.164	1,58×
	✗	1.508.605	
✓	✓	1.470.299	1,56×
	✗	2.295.773	

configurations perform identical arithmetic, the improvement results solely from avoiding intermediate memory transfers. This confirms that data movement between accelerator commands is a major contributor to the co-design cost.

We also measure the second implementation option, the optimized sampling of Section 3.2. Disabling rejection sampling on the co-design lowers the unmasked cost from 957.164 to 950.904 cycles and the masked cost from 1.470.299 to 1.464.064 cycles, a saving of less than 0.7% in both cases. Rejection sampling is therefore nearly free on the co-design, since the rejection loop runs on the sampling hardware of the accelerator, and we keep it enabled in all other measurements to stay faithful to the PASTA specification.

6.4 Profiling Breakdown

To see where the cycles are spent, we profile one encryption and group the cost by hardware block in Table 4. Here each block is timed separately, so the per-block reads add a small overhead and the total comes out a bit higher than the single end-to-end measurement of Table 2 (1.021.734 vs. 957.164 cycles unmasked). We use these numbers only to compare the share of each block. The arithmetic accelerator dominates in both designs, taking 992.718 cycles (97,16%) unmasked and 1.515.207 cycles (98,21%) masked, which shows that the matrix product is the main cost and the source of the speed-up. The Keccak block, which builds the public matrices and round constants, takes about 14.000 cycles in both designs, a small part of the total.

The masked configuration additionally invokes the masking accelerator. The masking accelerator, which refreshes the shares before each masked product, is not used in the unmasked design and takes 7.062 cycles in the masked design, about 0,46% of the total. The cost of masking therefore shows up mostly as extra work on the arithmetic accelerator, since each masked product runs the accelerator more often, and only to a small degree as the refreshing operations performed by the masking accelerator. Software overhead remains small in both configurations because the processor only issues accelerator commands.

The command-level profile further explains the sources of the observed speed-up. Table 5 gives the cost of the operations that the matrix product and the sampling use most, measured for a single call on the co-design and in software.

Table 4: Per-block cost breakdown of one PASTA-3 encryption. Block costs are measured separately, so the total slightly exceeds the end-to-end cost of Table 2; shares are relative to this total.

	Masking			
	✗		✓	
Block	**Cycles**	**Share**	**Cycles**	**Share**
Arithmetic accelerator	992.718	97,16%	1.515.207	98,21%
Keccak accelerator	14.420	1,41%	14.376	0,93%
Masking accelerator	0	0%	7.062	0,46%
Software	14.596	1,43%	6.154	0,40%
Total	1.021.734	100%	1.542.799	100%

Table 5: Per-call cost of the most-used operations in software and on the co-design, with the per-call speed-up.

Operation	**SW**	**Co-design**	**Speed-up**
Polynomial sampling	220.226	982	224,26×
Pointwise multiplication	3.329	430	7,74×
Pointwise addition	1.668	420	3,97×

A pointwise multiplication runs in 3.329 cycles in software against 430 cycles on the accelerator, a per-call speed-up of 7,74×, and a pointwise addition runs in 1.668 cycles against 420 cycles, a per-call speed-up of 3,97×. Polynomial sampling shows the largest gap, 220.226 cycles in software against 982 cycles on the accelerator, a per-call speed-up of 224,26×, because the rejection loop and the field reduction run on dedicated hardware. However, sampling is invoked only a few times per encryption; consequently, the frequently executed pointwise operations determine the overall speed-up. From the two pointwise numbers alone, one would expect the full encryption to speed up by somewhere between 3,97× and 7,74×. The measured speed-up without the memory optimization is 6,14×, which sits inside this range and matches the expectation. With the memory optimization enabled, however, the speed-up rises to 9,68×, above the per-operation range. The extra gain does not come from faster arithmetic, since the operations are the same, but from removing the load and store calls between them. In other words, the per-operation speed-up is set by the accelerator, and the memory optimization is what lets the co-design go beyond it. This indicates that, after arithmetic acceleration, memory traffic becomes the main remaining optimization target.

Table 6: Memory footprint of the four PASTA-3 designs, in bytes.

	Masking					
	✗			✓		
Design	**Text**	**Data**	**Total**	**Text**	**Data**	**Total**
SW	14.096	192	14.288	14.740	192	14.932
HW/SW	2.864	0	2.864	3.880	0	3.880

6.5 Hardware Cost and Memory Footprint

The co-design introduces no new hardware and reuses the existing RISQrypt accelerators in both the unmasked and masked implementations. Consequently, the two designs differ only in their code and data footprints, reported in Table 6. The total footprint is 2,864 bytes for the unmasked co-design and 3,880 bytes for the masked co-design, compared with 14,288 and 14,932 bytes for the corresponding software-only implementations. This reduction results from expressing most cipher operations as compact accelerator command sequences rather than complete software arithmetic routines.

6.6 Comparison with Prior Work

The original PASTA work [12] reports 17.041.380 cycles for one plain PASTA-3 encryption on an Intel Xeon E5-2699 v4, which shows that even a desktop CPU spends millions of cycles per block and motivates dedicated support on embedded clients. Although these platforms differ too much for a direct comparison, our embedded software baseline needs 9.267.385 cycles and the co-design brings this down to 957.164 cycles, well below the CPU reference.

Table 7 summarizes the evaluated designs on embedded platforms. [2] is the closest hardware reference. It uses a dedicated PASTA datapath, so it reaches much lower latency than our co-design. However, its FPGA core alone occupies about 65K LUTs and 256 DSPs, while our design adds no hardware on top of the existing platform. Our goal is different, since we do not add a PASTA-specific RTL block and we reuse the existing RISQrypt blocks for both the unmasked and masked designs. RISQrypt's hardware cost is also much smaller. Its unmasked platform uses about 11K LUTs and 9 DSPs, and the masked one about 26K LUTs and 15 DSPs. These resource requirements are substantially lower than those of the dedicated core, making the platform more suitable for resource-constrained devices. We also note that [2] offers no SCA protection.

[1] also presents a masked implementation of PASTA, albeit without hardware acceleration. Targeting a Cortex-M4, it reports 4,249,061 cycles for the unmasked implementation and 15,549,629 cycles for the masked implementation of a single PASTA-3 block. In comparison, our co-design requires 957,164 cycles in the unmasked setting, corresponding to a $4,44\times$ speed-up, and 1,470,299 cycles in the masked setting, corresponding to a $10,58\times$ speed-up. Since the

Table 7: Comparison with related implementations and our evaluated designs.

Work	SCA-Protected	Implementation style
[2]	✗	Dedicated PASTA hardware
[1]	✓	Software-only
This work	✓	HW/SW co-design

two implementations target different platforms, these figures should not be interpreted as a direct performance comparison. Consequently, we do not use [1] as the software baseline; instead, we evaluate our co-design against our own software implementation running on the same platform, as reported above.

7 Conclusion

In this paper, we presented a hardware/software co-design implementation of PASTA for hybrid homomorphic encryption. The design maps PASTA to the existing RISQrypt platform and reuses its arithmetic accelerator (NTT-Lite), Keccak accelerator, and masking accelerator (X2X Accelerator), instead of introducing a new PASTA-specific RTL accelerator. This shows that an embedded platform originally designed for lattice-based post-quantum cryptography can also support an HE-friendly cipher over $\mathbb{F}_p$ through software-controlled accelerator scheduling.

We also integrated first-order arithmetic masking into the co-design. The protected implementation keeps the PASTA state in shared form during the round computation and reuses the same hardware blocks as the unmasked implementation. As a result, masking is supported at the co-design level without changing the hardware architecture, and the masked multiplication passes a TVLA evaluation with 100 K traces on the target FPGA.

The implementation results show that the co-design provides a clear performance gain over software-only execution on the same platform. For one PASTA-3 block, the unmasked co-design achieves a 9,68× speed-up, while the masked co-design reaches a 9,85× speed-up over the corresponding software-only implementations. The co-design requires about 440 cycles/bit in the unmasked setting and 676 cycles/bit in the masked setting. The profiling results further show that the arithmetic accelerator dominates the execution time, while memory optimization is essential for removing unnecessary data transfers.

Future work includes extending the platform to other HE-friendly ciphers and comparing their performance and masking costs under the same co-design methodology.

8 Acknowledgments

This work was supported by the European Union's Horizon Europe research and innovation program through the PHASE IV AI Project under Grant 101095384.

References

1. Aikata, A., Czuprynko, M., Mušović, N., Salkić, E., Roy, S.S.: Attacking and securing hybrid homomorphic encryption against power analysis. In: 2026 Design, Automation & Test in Europe Conference (DATE). pp. 1–7 (2026). https://doi.org/10.23919/DATE69613.2026.11539375
2. Aikata, A., Sobrino, D.S., Roy, S.S.: Pasta on edge: Cryptoprocessor for hybrid homomorphic encryption. In: 2025 Design, Automation & Test in Europe Conference (DATE). pp. 1–7 (2025). https://doi.org/10.23919/DATE64628.2025.10992820
3. Albrecht, M.R., Rechberger, C., Schneider, T., Tiessen, T., Zohner, M.: Ciphers for MPC and FHE. In: Advances in Cryptology - EUROCRYPT 2015. vol. 9056, pp. 430–454. Springer (2015). https://doi.org/10.1007/978-3-662-46800-5_17
4. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (leveled) fully homomorphic encryption without bootstrapping. In: Proceedings of the 3rd Innovations in Theoretical Computer Science Conference. p. 309–325. ITCS '12, Association for Computing Machinery (2012). https://doi.org/10.1145/2090236.2090262
5. Cannière, C.D.: Trivium: A stream cipher construction inspired by block cipher design principles. In: Katsikas, S.K., López, J., Backes, M., Gritzalis, S., Preneel, B. (eds.) Information Security, 9th International Conference, ISC 2006, Samos Island, Greece, August 30 - September 2, 2006, Proceedings. vol. 4176, pp. 171–186. Springer (2006). https://doi.org/10.1007/11836810_13
6. Cassiers, G., Masure, L., Momin, C., Moos, T., Standaert, F.X.: Prime-field masking in hardware and its soundness against low-noise sca attacks. IACR Transactions on Cryptographic Hardware and Embedded Systems **2023**(2), 482–518 (Mar 2023). https://doi.org/10.46586/tches.v2023.i2.482-518
7. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic encryption for arithmetic of approximate numbers. In: International Conference on the Theory and Application of Cryptology and Information Security (2017), https://api.semanticscholar.org/CorpusID:3164123
8. Cho, J., Ha, J., Kim, S., Lee, B., Lee, J., Lee, J., Moon, D., Yoon, H.: Transciphering framework for approximate homomorphic encryption. In: Tibouchi, M., Wang, H. (eds.) Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part III. vol. 13092, pp. 640–669. Springer (2021). https://doi.org/10.1007/978-3-030-92078-4_22
9. Cosseron, O., Hoffmann, C., Méaux, P., Standaert, F.X.: Towards case-optimized hybrid homomorphic encryption: Featuring the elisabeth stream cipher. In: Advances in Cryptology – ASIACRYPT 2022. p. 32–67. Springer-Verlag, Berlin, Heidelberg (2022). https://doi.org/10.1007/978-3-031-22969-5_2
10. Daemen, J., Rijmen, V.: The Design of Rijndael: AES - The Advanced Encryption Standard. Information Security and Cryptography, Springer (2002). https://doi.org/10.1007/978-3-662-04722-4
11. Dobraunig, C., Eichlseder, M., Grassi, L., Lallemand, V., Leander, G., List, E., Mendel, F., Rechberger, C.: Rasta: A cipher with low anddepth and few ands per bit. In: Advances in Cryptology - CRYPTO 2018. vol. 10991, pp. 662–692. Springer (2018). https://doi.org/10.1007/978-3-319-96884-1_22
12. Dobraunig, C., Grassi, L., Helminger, L., Rechberger, C., Schofnegger, M., Walch, R.: Pasta: A case for hybrid homomorphic encryption. IACR Transactions on Cryptographic Hardware and Embedded Systems **2023**(3), 30–73 (Jun 2023). https://doi.org/10.46586/tches.v2023.i3.30-73

13. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. IACR Cryptol. ePrint Arch. **2012**, 144 (2012), https://api.semanticscholar.org/CorpusID:1467571
14. Gilbert Goodwill, B.J., Jaffe, J., Rohatgi, P.: A testing methodology for side-channel resistance validation. In: NIST non-invasive attack testing workshop. vol. 7, pp. 115–136 (2011)
15. Ha, J., Kim, S., Lee, B., Lee, J., Son, M.: Rubato: Noisy ciphers for approximate homomorphic encryption. In: Advances in Cryptology - EUROCRYPT 2022. Lecture Notes in Computer Science, vol. 13275, pp. 581–610. Springer (2022). https://doi.org/10.1007/978-3-031-06944-4_20
16. Jeon, Y., Erez, M., Orshansky, M.: Presto: Hardware acceleration of ciphers for hybrid homomorphic encryption (2025), https://arxiv.org/abs/2507.00367
17. Keccak Team: eXtended Keccak Code Package (XKCP). https://github.com/XKCP/XKCP, accessed: 2026-07-11
18. Méaux, P., Journault, A., Standaert, F., Carlet, C.: Towards stream ciphers for efficient FHE with low-noise ciphertexts. In: Advances in Cryptology - EUROCRYPT 2016. vol. 9665, pp. 311–343. Springer (2016). https://doi.org/10.1007/978-3-662-49890-3_13
19. Naehrig, M., Lauter, K., Vaikuntanathan, V.: Can homomorphic encryption be practical? In: Proceedings of the 3rd ACM Workshop on Cloud Computing Security Workshop. p. 113–124. Association for Computing Machinery, New York, NY, USA (2011). https://doi.org/10.1145/2046660.2046682
20. Niu, C., Wei, B., Huang, Z., Yang, Z., Hong, C., Wang, M., Wei, T.: Sok: Fhe-friendly symmetric ciphers and transciphering. IACR Transactions on Cryptographic Hardware and Embedded Systems **2025**(3), 583–613 (Jun 2025). https://doi.org/10.46586/tches.v2025.i3.583-613
21. Thakur, I., Karmakar, A., Li, C., Preneel, B.: A survey on transciphering and symmetric ciphers for homomorphic encryption. Cryptology ePrint Archive, Paper 2025/093 (2025), https://eprint.iacr.org/2025/093
22. Tosun, T., Ay, A.U., Norga, Q., Kundu, S., Yazıcı, M., Savaş, E., Verbauwhede, I.: RISQrypt: Fast, secure and agile hardware-software co-design for post-quantum cryptography. Cryptology ePrint Archive, Paper 2026/312 (2026), https://eprint.iacr.org/2026/312